HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: parrot

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Tìm hướng tấn công chính xác **3**

2.1 Thử payload để loại bỏ các hướng tấn công sai 3

3 Kiểm tra lại hướng tấn công và dự đoán OS server challenge dùng **4**

4 Sử dụng payload đặc biệt để lọc ra ngay file nghi vấn chứa flag **4**

5 Đọc file nghi vấn và tìm được flag **5**

1 Phân tích Challenge

Challenge cho chúng ta một đường link:

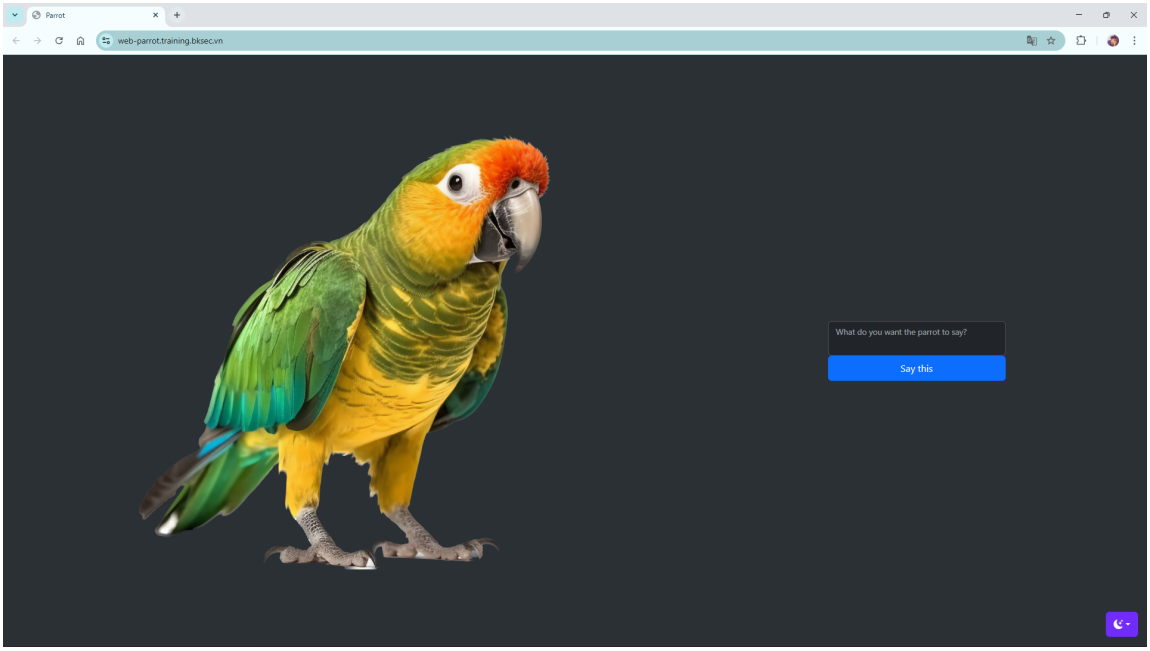
```
https://web-parrot.training.bksec.vn/
```

cùng với một đoạn gợi ý:

"A simple "repeat after me"challenge."

Với mô tả "repeat after me" (nhập gì nó lặp lại đó), bản chất là **user-controlled input** được phản hồi lại trong response → rất dễ tạo bề mặt *injection*. Vì vậy, Duc nhận định challenge này có thể thuộc các hướng như **XSS**, **SSTI**, **OS Command Injection** hoặc thậm chí là **SQL Injection**.

Trước tiên, **Duc** mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Nhìn vào giao diện, Duc nhận thấy trang web hiển thị một hình **con vẹt** (*parrot*) và một ô nhập liệu với gợi ý *"What do you want the parrot to say?"*. Bên dưới là nút *"Say this"*. Điều này cho thấy trang web có cơ chế nhận input từ người dùng và trả lại output theo đúng nội dung đã nhập (*repeat after me*).

Để xác định chính xác hướng tấn công (input được render như HTML, được đưa vào template, hay bị đưa vào command/query), Duc ném link vào *Burp Suite* để bắt đầu vọc vạch và phân tích request/response.

2 Tìm hướng tấn công chính xác

2.1 Thử payload để loại bỏ các hướng tấn công sai

Để loại bỏ các hướng tấn công không phù hợp, Duc lần lượt thử các payload theo từng nhóm lỗ hổng và quan sát phản hồi của trang web.

- **Kiểm tra XSS:** Duc nhập payload `<script>alert(1)</script>` để xem ứng dụng có render HTML/JavaScript hay không. Kết quả là trang web **không in gì ra cả**. Điều này cho thấy input có thể đã bị filter/escape mạnh theo hướng HTML, hoặc không được render trực tiếp ra DOM. Vì vậy, Duc tạm loại hướng tấn công **XSS**.

- **Kiểm tra SSTI:** Duc thử 2 payload phổ biến:

  – `{{7*7}}`
  – `${7*7}`

  Với `{{7*7}}`, trang web in **y nguyên** payload mà không tính toán gì cả. Thậm chí với `${7*7}`, trang web còn **không in gì**. Vậy Duc loại bỏ hướng tấn công **SSTI**.

- **Kiểm tra SQL Injection:** Duc nhập thử payload `' OR '1'='1`. Trang web phản hồi bằng việc in ra màn hình nội dung `OR 1=1` mà không có bất kỳ dấu hiệu lỗi SQL nào (không error message, không thay đổi logic). Do đó, Duc tiếp tục loại bỏ hướng tấn công **SQL Injection**.

  Sau các phép thử trên, hướng tấn công còn lại khả nghi nhất là **OS Command Injection**.

3 Kiểm tra lại hướng tấn công và dự đoán OS server challenge dùng

Với hướng tấn công tiềm năng nhất là **OS Command Injection**, Duc quyết định thử vận may với môi trường **Linux**. Duc nhập payload sau và nhấn *Enter*:

```
; echo 'DucTapCodeDao'
```

Kết quả là trang web đã in ra màn hình chuỗi `DucTapCodeDao`. Điều này chứng tỏ input của người dùng đã bị ghép trực tiếp vào một lệnh hệ thống, cho phép dùng ký tự phân tách lệnh (`;`) để thực thi lệnh bổ sung.

Vậy là không còn gì nghi ngờ nữa: challenge này khai thác **OS Command Injection**, và hệ điều hành phía server đang sử dụng là **Linux**.

4 Sử dụng payload đặc biệt để lọc ra ngay file nghi vấn chứa flag

Sau khi đã xác định được hệ điều hành của server là **Linux**, Duc quyết định tung một đòn "một mũi tên trúng hai con chim" bằng payload sau:

```
; ls -la / | grep -i "flag"
```

Payload này hoạt động như sau:

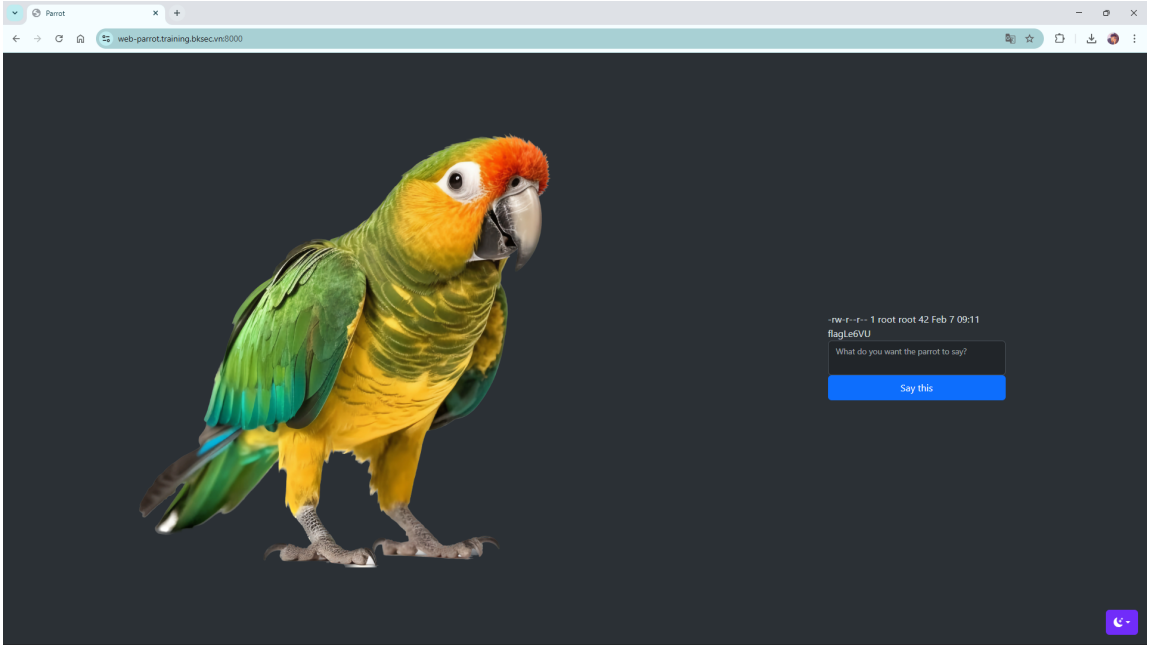
- Ký tự `;` dùng để **kết thúc** lệnh gốc mà server đang chạy, sau đó cho phép nối thêm lệnh mới.

- `ls -la /` liệt kê toàn bộ file/thư mục trong thư mục gốc `/`.

- Dấu pipe `|` chuyển output của `ls` làm input cho `grep`.

- `grep -i "flag"` tìm (không phân biệt hoa thường) các dòng có chứa chuỗi `flag`, từ đó lọc ra những file/thư mục có tên liên quan đến flag.

Và sau khi nhập payload này, Duc đã thu được "quả ngọt" như trong hình sau:



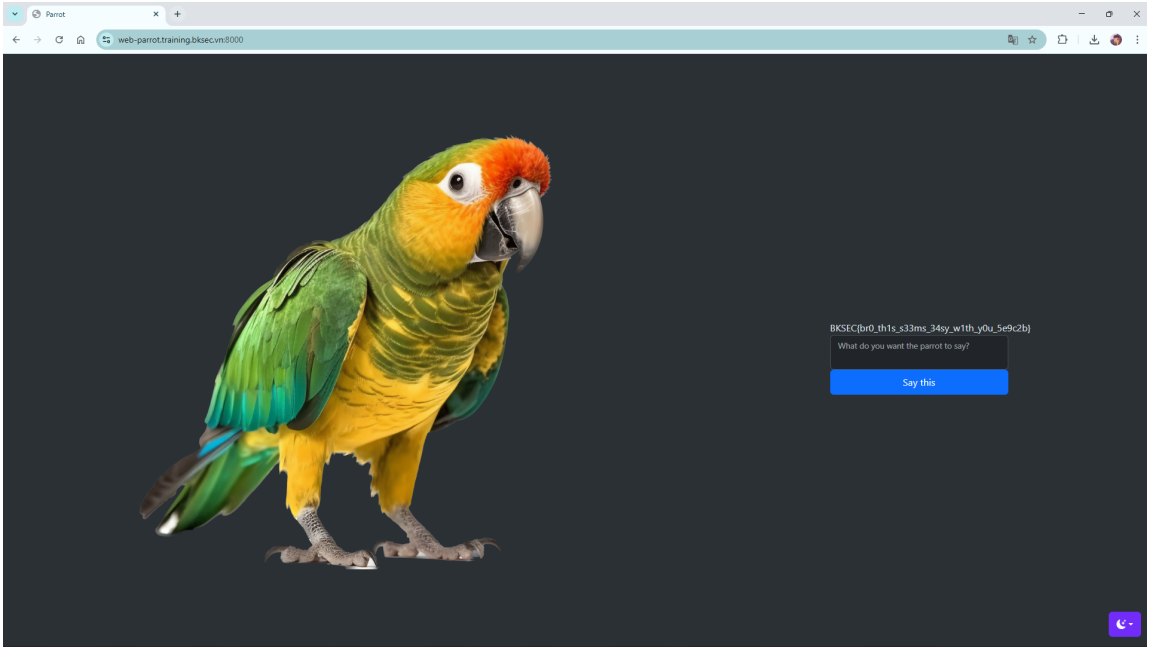
Hình 2: Kết quả thực thi payload `; ls -la / | grep -i "flag"` để dò file liên quan đến flag trong thư mục gốc.

5 Đọc file nghi vấn và tìm được flag

Khi mà đã tìm được tên file tiềm năng, việc còn lại bây giờ chỉ đơn giản là **đọc nội dung file**. Vì vậy, Duc tiếp tục chèn thêm một payload nữa để đọc file vừa tìm được:

```
; cat /flagLe6VU
```

Và trang web đã trả cho Duc kết quả như sau:



Hình 3: Thực thi payload `; cat /flagLe6VU` để đọc nội dung file flag và thu được flag.

Duc nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

`BKSEC{br0_th1s_s33ms_34sy_w1th_y0u_5e9c2b}`

Problem solved!